

ComSci 342 Homework 8

1. (8 pt) [Prolog for Numbers]

- a. (4 pt) Write a Prolog program to compute the sequence of Fibonacci numbers given its length.



```

1 fibonacci(0, []).
2 fibonacci(N, List) :-
3     N > 0,
4     fib_helper(N, 0, 1, List).
5
6 fib_helper(0, _, _, []).
7 fib_helper(N, A, B, [A|Rest]) :-
8     N > 0,
9     N1 is N - 1,
10    Next is A + B,
11    fib_helper(N1, B, Next, Rest).

```

- b. (4 pt) Write a Prolog program to check if a given integer i ($0 < i < 1000$) is a prime.



```

1 isprime(N) :-
2     integer(N),
3     N > 1,
4     \+ has_factor(N, 2).
5
6 has_factor(N, F) :-
7     F * F =< N,
8     N mod F =:= 0.
9
10 has_factor(N, F) :-
11     F * F =< N,
12     F1 is F + 1,
13     has_factor(N, F1).

```

2. (10 pt) [Prolog for List]

a. (5 pt) Write a Prolog program to filter a list containing a given number N

 RLYQgzcB ✕ +

```
1 filterlist([], _, []).
2
3 filterlist([N|T], N, Z) :-
4     filterlist(T, N, Z).
5
6 filterlist([H|T], N, [H|Z]) :-
7     H \= N,
8     filterlist(T, N, Z).
9
```

b. (5 pt) Write a Prolog program to sort a list of numbers in ascending order.

 RLYQgzcB ✕ +

```
1 sortList([], []).
2
3 sortList([H|T], Sorted) :-
4     sortList(T, SortedTail),
5     insert_sorted(H, SortedTail, Sorted).
6
7 insert_sorted(X, [], [X]).
8
9 insert_sorted(X, [H|T], [X,H|T]) :-
10     X <= H.
11
12 insert_sorted(X, [H|T], [H|R]) :-
13     X > H,
14     insert_sorted(X, T, R).
15
```

3. (6 pt) [Prolog for Word Puzzle] Write a Prolog program to solve a word puzzle.



RLYQgzcB



```
1 poem([
2     ever, and, again, i, open, my, door, and, look,
3     out, on, the, darkness, my, friend
4 ]).
5
6 word_length(Word, Len) :-
7     atom_chars(Word, Chars),
8     length(Chars, Len).
9
10 letter_at(Word, Pos, Letter) :-
11     atom_chars(Word, Chars),
12     nth1(Pos, Chars, Letter).
13
14 puzzle_solution(W1, W2) :-
15     poem(Words),
16     member(W1, Words),
17     member(W2, Words),
18     W1 \= W2,
19
20     word_length(W1, 8),
21     word_length(W2, 4),
22
23     letter_at(W1, 6, Letter),
24     letter_at(W2, 3, Letter).
25
```



puzzle_solution(W1, W2).

W1 = darkness,

W2 = ever

W1 = darkness,

W2 = open

false

4. (15 pt) [Prolog for Mini-Sudoku] Write a Prolog program to solve a given Mini-Sudoku instance, which is a simplified version of the popular Sudoku puzzles.

```

1 digit(1).
2 digit(2).
3 digit(3).
4 digit(4).
5 digit(5).
6
7 sudoku(M) :-
8     M = [
9         [X1, 2, X6, 4, 5],
10        [2, X2, 4, 5, 1],
11        [3, 4, X3, 1, 2],
12        [4, 5, 1, X4, 3],
13        [X7, 1, 2, 3, X5]
14    ],
15    fill_digits([X1, X2, X3, X4, X5, X6, X7]),
16    rows_valid(M),
17    transpose(M, T),
18    rows_valid(T).
19
20 fill_digits([]).
21 fill_digits([H|T]) :-
22     digit(H),
23     fill_digits(T).
24
25 rows_valid([]).
26 rows_valid([R|Rs]) :-
27     unique(R),
28     rows_valid(Rs).
29
30 unique([]).
31 unique([H|T]) :-
32     \+ member(H, T),
33     unique(T).
34
35 transpose([_|_], []).
36 transpose(M, [Col|Cols]) :-
37     first_column(M, Col, Rest),
38     transpose(Rest, Cols).
39
40 first_column([], [], []).
41 first_column([[H|T]|Rows], [H|Hs], [T|Ts]) :-
42     first_column(Rows, Hs, Ts).

```



sudoku(M).

M = [[1, 2, 3, 4, 5], [2, 3, 4, 5, 1], [3, 4, 5, 1, 2], [4, 5, 1, 2, 3], [5, 1, 2, 3, 4]]

X1 = 1, X2 = 3, X3 = 5, X4 = 2, X5 = 4, X6 = 3, X7 = 5.

5. (13 pt) [Prolog for Parsing]

- a. (10 pt) Write a Prolog parser for the grammar for VarLang. This parser should be capable of deciding whether a given program belongs to the language.

```
RLYQgzcB +
1 program([]).
2 program(Tokens) :-
3     exp(Tokens, []).
4
5 exp([N|Rest], Rest) :-
6     number(N).
7
8 exp([Id|Rest], Rest) :-
9     valid_identifier(Id).
10
11 exp(['(', Op|Rest], FinalRest) :-
12     operator(Op),
13     exp(Rest, Rest1),
14     exp(Rest1, Rest2),
15     Rest2 = ['')|FinalRest].
16
17 exp(['(', let, '('|Rest], FinalRest) :-
18     bindings(Rest, Rest1),
19     Rest1 = ['')|Rest2],
20     exp(Rest2, Rest3),
21     Rest3 = ['')|FinalRest].
22
23 operator(+).
24 operator(-).
25 operator(*).
26 operator(/).
27
28 bindings(['(', Id|Rest], FinalRest) :-
29     valid_identifier(Id),
30     exp(Rest, Rest1),
31     Rest1 = ['')|Rest2],
32     more_bindings(Rest2, FinalRest).
33
34 more_bindings(['(', Id|Rest], FinalRest) :-
35     valid_identifier(Id),
36     exp(Rest, Rest1),
37     Rest1 = ['')|Rest2],
38     more_bindings(Rest2, FinalRest).
39
40 more_bindings(Rest, Rest).
41
42 valid_identifier(Id) :-
43     atom(Id),
44     Id \= '(',
45     Id \= ')',
46     Id \= '+',
47     Id \= '-',
48     Id \= '*',
49     Id \= '/',
50     Id \= let.
```

- b. (3 pt) Write a Prolog query that determines whether a given program is syntactically correct in VarLang, pertaining to the grammar defined above. Include an example program in your query.

 `program(['(', 'let', '(', '(', m, 10, ')', ')', '(', '+', m, 2, ')', ')']).`

`true`

`?- program(['(', 'let', '(', '(', m, 10, ')', ')', '(', '+', m, 2, ')', ')']).`